

E01 · training이란 무엇인가 —무엇이 언제움직이나

우리가 아는 serving 에서 모델 가중치는 read-only 상수다. checkpoint 파일을 mmap 해 GPU 에 올리고 나면 그 뒤로는 읽기만 한다. 요청이 오고 가도, KV cache 가 찼다 비워져도, 가중치는 한 바이트도 바뀌지 않는다. 이 불변식위에 우리가 아는 서버의 거의 전부가 서 있다 — batch 안의 서로 다른요청이 같은 가중치를 공유할 수 있는 것도, 노드를 늘려도 답이 같은 것도, rollback 이 파일 하나를 되돌리는 일인 것도 전부 여기서 나온다.

sleep-time compute 은 그 불변식을 정면으로 건드리는 라인이다. 이 라인의논문들은 “질의가 오기 전 유희 시간에 무언가를 계산해서 어딘가에 써둔다” 고 말하는데, 그 “어딘가” 중 하나가 바로 가중치다. 그리고 가중치에 쓴다는 것은 training 을 한다는 뜻이다. 그래서 이 시리즈의 첫 권이 training 이다.

이 권의 목표는 training 을 할 줄 아는 것이 아니라 읽을 줄 아는 것이다. 논문 부록에서 “steps 100, effective batch size 32, LR 5e-6, LoRA rank 64” 라고 적힌 표를 만났을 때 그것이 몇 토큰이고 몇 FLOPs 이며 디스크에 몇 MB 를 남기는지 그 자리에서 셀 수 있으면 이 권은 제일을 한 것이다. 증명은 하나도 하지 않는다. 유도가 필요한 자리는 본문장으로 넘긴다.

1. 세 개의 서랍 — 무엇이 움직일 수있는가

먼저 좌표를 놓는다. 이 라인이 “기억한다” , “써 둔다” 고 말할 때 실제로쓰이는 자리는 세 곳뿐이다. 논문을 읽을 때 가장 먼저 판정해야 하는 것이 “이 논문은 셋 중 어느 서랍을 여는가” 다.

기호 풀이 Θ (세타) = **slow weights**. checkpoint 파일에 들어 있는가중치 전체이고, 서버 중에는 상수다. W = **fast weights**. 요청이 살아 있는 동안만 존재하며 토큰마다 갱신되는 작은 상태로, KV cache 가 놓인 자리와 같은 수명이다. E = **external store**. 모델바깥의 텍스트 · 벡터 · 그래프 기억이며, 읽을 때는검색해서 프롬프트에 붙인다.

세 서랍은 수명도 다르고 비용의 성격도 다르다. E 에 쓰는 것은데이터베이스에 행을 하나 넣는 일이고, W 에 쓰는 것은 세션 상태를갱신하는 일이며, Θ 에 쓰는 것은 **배포 아티팩트를 바꾸는일**이다. 마지막 것만이 파일을 새로 만들어 낸다. 그리고 마지막 것만이 training 이다.

이 권은 Θ 서랍만 다룬다. W 와 E 는 뒤 권의 몫이다. 다만여기서 한 문장만 못 박아 둔다 — Θ 에 쓰는 절차는 전부 **training** 이고, **training** 의 문법은 **pre-training** 이든 유희시간의 **sleep** 라운드든 글자 그대로 같다. 다르게 만드는 것은 세 가지뿐이며, 그 셋이 이권의 뼈대다: 데이터가 어디서 오는가, 무엇을 목표로 삼는가, Θ 의어느 부분이 움직이는가.

자세한 것은 ch01 § 01.2, ch02 § 02.1.

2. 학습이란 손잡이를 돌려 “틀린 정도” 를 줄이는일

Θ 에는 수십억 개의 숫자가 들어 있다. 이것을 손잡이라고 부르자. 학습이란 이 손잡이들을 조금씩 돌려, **틀린 정도**를 나타내는 숫자하나를 낮추는 반복 작업이다.

비유 loss는 “지금 이 모델이 얼마나 틀렸나”를 재는 점수판 하나다. 학습 = 점수판 숫자를 내리려고 수십억 개 손잡이를 아주 조금씩 돌리는 일. **이 비유가 깨지는 곳** — 손잡이들은 서로 독립이 아니다. 하나를 돌리면 나머지 전부의 최적 위치가 함께 움직인다. 그래서 “손잡이를 하나씩 최적으로 맞춘다”는 그림은 성립하지 않고, 전부를 동시에 조금씩 옮기는 방법밖에 없다.

한 번의 학습 step은 언제나 네 단계다.

1. **forward pass** — 데이터를 모델에 통과시켜 출력을 낸다. 커널구성이 우리가 아는 prefill과 똑같다 (GEMM과 attention의 연속).
2. **loss 계산** — 그 출력을 점수 하나로 줄인다. 비용은 사실상 0이다.
3. **backward pass** — 그 점수를 기준으로 “각 손잡이를 어느 쪽으로 얼마나 돌려야 하나”를 계산한다. FLOPs는 forward의 약 2배다.
4. **optimizer step** — 3에서 나온 지시를 읽어 손잡이를 실제로 돌린다.

3번이 계산하는 값을 gradient라고 부른다. 손잡이마다 하나씩 붙는 숫자이고, “이 손잡이를 아주 조금 올리면 점수가 얼마나 나빠지나”라는 민감도다. 부호와 크기를 함께 담는다.

$$\Theta \leftarrow \Theta - \eta_{\Theta} \cdot \nabla_{\Theta} \mathcal{L}$$

기호 풀이 $\mathcal{L}$ = loss, 틀린 정도를 나타내는 숫자 하나. $\nabla_{\Theta} \mathcal{L}$ = gradient, Θ 와 똑같은 크기의 텐서이고 손잡이별 민감도가 들어 있다. η_{Θ} (에타) = **learning rate**, 한 걸음의 보폭. 아래첨자 Θ 는 “느린 쪽 손잡이를 옮기는 보폭”이라는 뜻이며, 뒤 권에서 나올 W 쪽 보폭 η_t 와 반드시 구분한다.

이 식이 말하는 것은 “민감도의 반대 방향으로 보폭 η_{Θ} 만큼 손잡이를 옮겨라”이다. 반대 방향인 이유는 점수를 내리고 싶기 때문이다. 이 한 줄이 이 권에 나오는 모든 절차의 공통 문법이다 — pre-training 도, fine-tuning 도, sleep 라운드도 전부 이식을 정해진 횟수만큼 반복하는 것이다.

비유 안개 낀 산에서 발밑 경사만 느끼며 내려가는 상황. gradient는 “지금 자리에서 어느 쪽이 오르막인가”를 알려주는 화살표고, 내려가려면 화살표반대로 정해진 보폭만큼 한 걸음 옮기면 된다. **이 비유가 깨지는 곳** — 산은 가만히 있지 않는다. 매 step 다른 데이터 묶음을 보므로 지형자체가 매번 조금씩 바뀐다. 우리가 내려가는 것은 고정된 산이 아니라 **평균적으로만 같은 모양인 산**이다.

여기서 우리 세계의 loop와 나란히 놓아 보면 낯선 칸이 정확히 두 개라는 것이 보인다.
표 1 — serving loop와 training loop

항목	serving loop (우리 세계)	training loop (이 권)
가중치	read-only	매 step 읽고-고치고-쓰기
반복 단위	요청 / 토큰	step / batch
지속되는 상태	KV cache (요청 수명)	optimizer 상태 (run 전체 수명)

항목	serving loop (우리 세계)	training loop (이 권)
주 커널	forward GEMM + attention	forward GEMM + backward GEMM 2 종 + elementwise pass
roofline	prefill=compute-bound, decode=bandwidth-bound	forward/ backward=compute-bound, optimizer step=bandwidth-bound

낮선 칸은 **backward pass**와 **optimizer 상태** 둘뿐이다. backward는 4 단계 중 3 번이고, optimizer 상태는 4 번이 손잡이를 돌릴 때참고하는 내부 메모리다. 실무의 기본값인 AdamW는 손잡이 하나당 두 개의숫자를 들고 있다 — gradient의 평균과 gradient 제곱의 평균이다. 이 둘이있어야 손잡이마다 보폭을 다르게 줄 수 있다.

시스템 모델링 관점 training step은 우리가 아는 두 커널의 조합으로 완전히 분해된다. forward/backward는 **prefill**을 다했고 (두꺼운 GEMM, compute-bound), optimizer step은 **decode**를 다했다 (거대한 상태를한 번 훑는 elementwise pass, bandwidth-bound). 그리고 training이 serving보다 HBM을 몇 배 먹는 이유의 절반이 optimizer 상태다. 7B 모델을 bf16 가중치 + fp32 master + Adam 메모리 두 벌로 두면 손잡이당 14 byte, gradient와 activation을 세기도 전에 약 98 GB가 상주한다. “학습은 계산이 비싸다”는 인상은 절반만 맞다. 비싼 쪽은 자주메모리다.

자세한 것은 ch02 § 02.2.

3. 비용의 자 — 토큰 하나에 몇 FLOPs인가

레짐을 비교하려면 크기를 재는 자가 필요하다. 네 단어를 여기서 못 박는다.

기호 풀이 **step** = 위 식을 한 번 적용하는 것. Θ 가 움직이는 최소사건이며 개수를 n_{step} 으로 쓴다. **batch** = 한 step에서 gradient를 평균하는 표본 묶음. 시퀀스 개수를 N_b 로 쓴다. **epoch** = 데이터셋 전체를 한 번통과하는 것. **token budget** D = 그 run이 소비한 총 토큰 수. 그리고 N = 모델의 파라미터 개수, L = 시퀀스 길이 (층 수는 L_{layer} 로 쓴다. 이 책은 맨 L 을 층 수 의미로 쓰지않는다).

$$D = n_{\text{step}} \times N_b \times L$$

이 식이 말하는 것은 “총 소비 토큰 = step 수 × step 하나가 먹는토큰”이라는 곱셈 하나다. epoch은 여기서 유도되는 값일 뿐이다 — D 를데이터셋 크기로 나눈 비율이다. 데이터셋이 예산보다 크면 epoch은 1보다작아지고, 그때 “몇 epoch 돌렸다”는 말은 아무것도 설명하지 못한다.

논문 표에 적히는 batch 값은 대개 가속기 하나가 한 번에 밀어 넣은 개수가 아니라 **effective batch size**, 즉 “한 번의 갱신에 gradient가 평균되어 들어간 시퀀스의 총수” 다. 이 값은 가속기당 micro-batch $\times$ data-parallel 랭크 수 $\times$ gradient accumulation 횟수의 곱이다. 셋 중 어느조합으로 만들었든 D 도 FLOPs도 같다 — 바뀌는 것은 벽시계 시간과 HBM 점유뿐이다. 그래서 논문이 하드웨어를 밝히지 않아도 이 절의 회계는 성립한다.

이제 FLOPs로 옮긴다. 우리가 decode에서 쓰던 감각이 그대로 재취업한다.

$$\text{FLOPs} \approx 6ND \quad (\text{full fine-tuning}), \quad \text{FLOPs} \approx 2ND \quad (\text{생성만})$$

이 식이 말하는 것은 “학습은 같은 토큰에 생성의 3배를 쓴다” 이다. 6은 $2+4$ 로 갈린다. forward가 파라미터당 토큰당 곱셈-덧셈 한 쌍, 즉 2 FLOPs이고 — 우리가 decode한 토큰의 비용을 $2N$ 으로 잡던 그 값이다 — backward가 그 2배인 4다. backward의 4는 다시 둘로 갈린다: 오차를 앞층으로 전파하는 데 2, 손잡이의 gradient를 만드는 데 2.

이 분해가 실무에서 쓸모 있는 이유는 하나다. Θ 의 일부만 갱신하는 레짐에서는 **손잡이 gradient를 만드는 항이 통째로 빠져** $6ND$ 가 $4ND$ 로 내려간다. 그리고 유휴 시간에 모델이 텍스트를 생성만 하는 구간은 forward 뿐이므로 $2ND$ 다. 경로 사이 비용 비교의 첫 자릿수가여기서 이미 갈린다.

핵심 이 절의 두 식이 이 권의 계산 도구 전부다. 논문 부록에서 steps·batch·모델 크기 셋을 찾으면, 그 논문이 침묵하더라도 그 run의 토큰 수와 FLOPs는 되살아난다. 이 복원 절차를 corpus 전체에 기계적으로 돌리는 것이 이 책 3부의 작업이다.

자세한 것은 ch02 § 02.3.

4. 레짐 지도 — 하나의 checkpoint가 태어나기까지

우리가 서빙 엔진에 올리는 checkpoint 파일 하나는 여러 공정을 순서대로 통과한 결과물이다. 공정 하나를 **레짐 (regime)** 이라고 부른다. 레짐은 셋으로 식별된다: 데이터가 어디서 오는가, 목적함수가 무엇인가, Θ 의 어느 부분집합이 움직이는가.

오늘 우리가 서빙하는 instruct 계열 checkpoint 하나는 최소한 이 사슬을 통과했다.

무작위 초기값 $\rightarrow$ **pre-training** $\rightarrow$ **SFT** $\rightarrow$ **RLHF / RLVR** $\rightarrow$ (배포) $\rightarrow$ **continual**

각 화살표에서 checkpoint가 한 번씩 저장되고, 앞 레짐의 산출물이 뒤 레짐의 초기값이 된다. 앞의 세 화살표는 배포 **전에** 한 번씩 지나가고, 마지막 화살표만 배포 **후에** 반복된다.

핵심 이 라인이 사는 곳은 마지막 화살표 하나뿐이다. 앞의 셋을 이 권이 설명하는 이유는 그 자체가 목적이어서가 아니라, **마지막 화살표가 앞의 것들을 도구로 재사용하기 때문**이다. sleep 라운드 하나를 열어 보면 그안에 SFT와 RL과 distillation이 들어 있다.

비유 레짐 사슬은 건물 짓기와 인테리어의 관계다. pre-training이 골조를 세우고, 뒤의 레짐들은 이미 선 골조 위에서 마감을 바꾼다. **이 비유가 깨지는 곳** — 인테리어는 벽을 옮기지 않지만 full fine-tuning은 벽도 옮긴다. 그리고 그 결과물은 “고친 부분”이 아니라 **건물 한 채가 통째로 새로** 나온다. 이 사실이 뒤 § 8의 350GB를 만든다.

자세한 것은 ch02 § 02.1 과 § 02.8 의 표 2-1.

5. pre-training — 한 번만 지불되는 가장 큰비용

pre-training 은 무작위로 초기화된 Θ 를 대규모 미분류텍스트 위에서 “다음 토큰 맞히기” 를 목적으로 갱신하는 레짐이다. Θ 전체가 움직이고, 한 checkpoint 의 생애에서 D 가 압도적으로큰 유일한 레짐이다.

데이터는 주어진 것이고 사람이 라벨을 달지 않았으며, 정답은 데이터자신이다 — 다음 토큰이 곧 정답이다. 이 자기지도 성질 하나가 D 를인터넷 규모까지 밀어 올릴 수 있게 만든 이유 전부다.

구조적 특징 두 개만 기억하면 된다.

첫째, **epoch 이 대체로 1 근처다**. 즉 모델은 대부분의 토큰을 한번씩만 본다. 같은 데이터를 두 번 보지 않으므로 “잊었다” 를 관측할 기회자체가 적고, 그래서 뒤 권에서 다룰 forgetting 문제가 pre-training 안에서는 잘 드러나지 않는다.

둘째, **훈련 지표와 평가 지표가 같은 양이다**. 목적함수가 곧지표이므로 따로 떼어 둔 코퍼스의 점수를 그대로 읽으면 run 이 정상인지판정된다. 이것은 pre-training에만 있는 사치다. 뒤의 모든 레짐에서 loss 가내려가는 것과 능력이 오르는 것은 **별개의 사건**이 되며, 그 첫사례가 다음 절이다.

systems 접점은 비대칭이다. pre-training 은 이 책에 등장하는 어떤 레짐보다여러 자릿수 비싸고, 그 비용의 대부분이 **한 번만** 지불된다. 이후의모든 레짐은 그 위에 얹히는 작은 증분이다.

숫자로 세어 보면 이 비대칭의 크기가 보인다. 1.7B 모델에서 “steps 100, effective batch 32” 인 유희 시간 갱신 하나를 잡자 (시퀀스 길이 1024 는 이책이 고른 가정값이다). § 3 의 곱셈으로 $D = 100 \times 32 \times 1024 \approx 3.28 \times 10^6$ 토큰이고, $6ND = 6 \times 1.7 \times 10^9 \times 3.28 \times 10^6 \approx 3.34 \times 10^{16}$ FLOPs 다. 같은 모델의 pre-training 을 10^{12} 토큰으로 잡으면 1.02×10^{22} FLOPs 이므로, **유희 시간 갱신 한 번은 pre-training 의 약 3×10^{-6} 이다**. 하루 한 번씩 3 년을 돌려도 pre-training 한 번의 0.4%에 못 미친다.

핵심 이 비대칭이 “유희 시간에 모델을 갱신한다” 는 발상을 애초에 성립시킨다. 큰 비용은 이미 지불되었고, 우리가 논쟁하는 것은 그 위의 작은 증분이다.

자세한 것은 ch02 § 02.4, 계산의 전개는 ch02 의 Worked micro-example.

6. SFT — 마스크 한 장으로 정의되는레짐

supervised fine-tuning(SFT) 은 (입력, 원하는 출력) 쌍 위에서**출력 토큰에 대해서만** 다음 토큰 맞히기를 시키는 레짐이다. 데이터는 사람이 쓴 시연이거나 다른 모델의 출력이고, D 는 pre-training 보다 여러 자릿수 작다.

수식 수준에서 pre-training 과의 차이는 단 하나, **loss mask** 다. 시퀀스 전체에 점수를 매기는 대신 입력 (prompt) 구간을 가리고 출력 구간만 loss 에 넣는다. 목적함수를 바꾸는 비용이 사실상 0 이라는 말의 실감이 여기있다 — 레짐 하나가 마스크 텐서 한 장으로 정

의된다. 이 라인의 논문 중에는이 마스크와 보폭과 epoch 수를 **모델이 스스로 써 넣게** 만드는 것도있다.

SFT 는 원래 지식을 주입하려고 만든 레짐이 아니다. 설계 의도는 이미 Θ 안에 있는 능력을 원하는 출력 형식으로 꺼내는 것이다. 그런데이 라인의 논문들은 SFT 를 지식 주입 수단으로 쓴다. 그 용도 전용이무해하지 않다는 증거가 corpus 안에 이미 있다 — Qwen3-1.7B 에서 SFT 는 AIME-24 를 49.8 에서 47.3 으로, HMMT-25 를 25.7 에서 22.9 로 **내린다** [LM Need Sleep Table 2].

주의 SFT 를 돌렸다는 사실 자체는 능력이 올랐다는 증거가 아니다. 같은 표에서다른 절차들이 오르는 동안 SFT 만 내려갔다. 논문이 “fine-tuning 했으므로지식이 들어갔다” 고 쓰면 그 문장은 근거가 아니라 주장이다. 목적함수가무엇이었는지, 평가가 무엇을 잴는지를 함께 보아야 한다.

systems 접점은 크기다. SFT run 하나의 D 는 서버 노드 몇 대의 하루치처리량과 같은 자릿수다. 이 라인이 SFT 를 기본 도구로 삼는 이유는 성능이아니라 **예산이 맞기 때문이다**. 자세한 것은 ch02 § 02.5.

7. RL — 데이터를 run 안에서 만드는레짐

RLHF 는 사람의 선호로 학습된 별도 채점 모델이 매기는 점수를최대화하는 레짐이고, RLVR 은 그 점수를 학습된 모델이 아니라**프로그램으로 검증 가능한 정답 여부**가 매기는 레짐이다.

두 레짐의 정의적 특징은 보상이 아니라 데이터다. 지도학습 레짐에서 batch 는 run 바깥에서 주어진다. RL 레짐에서 batch 는 **지금 이순간의 Θ 가 생성한 rollout**이다. 즉 데이터 슬롯이 파라미터의함수가 되었다.

여기에 따름정리가 하나 붙는다. Θ 가 움직이는 순간 이전 step 에서만든 rollout 은 더 이상 현재 모델의 표본이 아니다. 그래서 RL 레짐은 “생성한 데이터를 몇 step 까지 재사용할 것인가” 라는, 지도학습에는 존재하지않는 결정을 반드시 내려야 한다. 재사용하지 않으면 매 step 생성 비용을다시 치르고, 재사용하면 데이터와 모델이 어긋난다. 같은 결정이 유희 시간갱신에도 그대로 나타난다.

“얼마나 드나” 의 답이 여기서 뒤집힌다. 진짜 비용은 step 수가 아니라 **step 당 생성량**이다. 생성이 붙은 레짐은 step 하나의 값이 다르며, corpus 의 한 논문은 자기 sleep step 하나가 SFT step 하나의 4 배라고보고한다 [LM Need Sleep App. B.5]. 그 초과분이 들어갈 회계 칸이 § 3 의 $2ND$ 항이다.

시스템 모델링 관점 이것이 이 권에서 가장 강한 systems 접점이다. **RL run 은 한 job 안에서 두 rooeline 레짐을 교대시킨다**. rollout 을 만드는 구간은 batch 가얇고 KV cache 를 순회하는 bandwidth-bound decode 이고, gradient 를 계산하는구간은 두꺼운 GEMM 의 compute-bound prefill 이다. 지도학습 run 에는 후자만있었다. 우리가 서버에서 익힌 “prefill 과 decode 를 섞어 스케줄링하는문제”가 training 클러스터 안으로 그대로 들어온 것이다. 그리고 유희 시간갱신이 이 형태를 물려받는 순간 결론이 하나 따라 나온다 — **sleep job 은 training 하드웨어가 아니라 wake 트래픽과 같은 하드웨어, 같은병목을 놓고 경쟁한다**.

비유 wake와 sleep의 관계는 낮에 매장을 열고 밤에 재고를 정리하는 것과 같다. 손님이 없는 시간을 써서 다음 날의 응답을 빠르고 정확하게 만든다. 이 비유가 깨지는 곳 — 재고 정리는 창고에서 하지만 sleep 라운드는 **매장 바닥에서** 한다. 위 박스가 말하는 그대로, 생성구간이 wake와 똑같은 자원을 먹기 때문이다. 유희 시간이 정말로 공짜인 클러스터에서만 이 비유가 성립한다.

자세한 것은 ch02 § 02.6, 개별 알고리즘은 ch05.

8. full fine-tuning — 왜 이것이 곧 모델재배포인가

이제 “ Θ 의 어느 부분이 움직이는가” 칸으로 들어간다. **full fine-tuning**은 사전학습된 Θ_0 (출발점이 되는 원래 가중치 전체, 아래첨자 0은 “아직 아무것도 쓰지 않은 상태”라는 뜻이다)를 새 목적함수의 gradient로 옮기는 절차다. 움직임에 구조적 제약이 없다는 것이 정의의전부다.

먼저 왜 필요한지부터 정한다. 문맥에 예시를 몇 개 넣는 것으로는 대체되지 않기 때문이다. GPT-3 175B에서 few-shot과 fine-tuning의 격차는 MNLI-m 40.6 대 89.5, RTE 69.0 대 85.4다 [lora Table 8]. 40점 이상의 격차는프롬프트 기법으로 메울 수 있는 종류가 아니다. **가중치에 쓰는 것과문맥에 쓰는 것은 같은 일이 아니다.**

문제는 대가다. full fine-tuning의 산출물은 Θ_0 와 **크기가같은** 새 checkpoint다. GPT-3 175B의 checkpoint는 350GB이므로 task 하나마다 350GB가 새로 생긴다 [lora § 4.2]. task가 100개면 약 35TB다.

핵심 full fine-tuning은 **모델 재배포**다. 배포 파이프라인, 이미지 크기, rollback 전략이 전부 달려 온다. 그리고 task마다 아티팩트가 하나씩생기며, 그 아티팩트들은 서로 다른 GPU에 올라가야 한다.

쓰기 시점의 비용도 배포 비용과 별개로 존재한다. GPT-3 175B의 full fine-tuning은 VRAM 1.2TB를 쓴다 [lora § 4.2]. 이 숫자의 출처를 정확히 아는 것이 다음 절의 열쇠다 — 지배하는 것은 파라미터 자체가 아니라 **optimizer 상태와 gradient**다. § 2에서 본 손잡이당 두 벌의메모리가 여기서 청구서로 돌아온다. 따라서 학습되는 파라미터 수를 줄이면메모리는 파라미터 수보다 **가파르게** 줄어든다.

자세한 것은 ch03 § 03.1.

9. LoRA — 변화량에 rank를걸다

LoRA(Low-Rank Adaptation)는 Θ 의 증분을 두 저차원행렬의 곱으로 제약하고 그 두 행렬만 학습하는 방법이다. 즉 Θ 전체를 옮기는 대신 **아주 얇은 부분공간 안에서만** 옮긴다.

$$y = \Theta_0^{(m)} x + \frac{\alpha_{\text{LoRA}}}{r} B^{(m)} A^{(m)} x$$

기호 풀이 $\Theta_0^{(m)}$ = 얼려 둔 원래 weight 행렬 하나 (m 은 몇 번째행렬인지를 가리키는 인덱스). $A^{(m)}$ 과 $B^{(m)}$ = 새로 학습하는 두 행렬. B 는 $d_{\text{out}} \times r$, A 는 $r \times d_{\text{in}}$ 모양이다. r = **rank**, 두 행렬이 만나는 내부 폭이며 이 방법의 유일한 용량 knob 이다. $\alpha_{\text{LoRA}} = 8 \cdot 16 \cdot 32$ 같은 상수배율.

이 식이 말하는 것은 “얼린 경로 하나와 얇은 경로 하나가 **나란히** 붙고 출력에서 더해진다” 이다. 나란히 붙는다는 것이 결정적이다 — 층이 깊어지지 않으므로 순차 처리가 강제되지 않고, 두 항이 같은 입력에 대한 선형사상이므로 배포 전에 하나로 합칠 수 있다. 이합치기를 **merge** 라고 부른다.

초기화가 이 방법의 성격을 정한다. A 는 무작위, B 는 0 으로 시작하므로 학습 시작 시점의 증분이 정확히 0 이다. **쓰기가항등원에서 출발한다** — 쓰기 전 모델이 한 치도 손상되지 않는다는 이성질은 이 라인의 논문들이 반복해서 증명한다.

비유 LoRA delta 는 원본 사진을 그대로 두고 그 위에 보정 레이어 한 장을 얹는 것과 같다. 원본은 건드리지 않고, 필요 없으면 레이어만 빼면 된다. **이 비유가 깨지는 곳** — 보정 레이어는 원하는 어떤 보정도 할 수 있지만 LoRA 는 rank r 짜리 부분공간 안의 변화만 표현할 수 있다. 표현할 수 없는 변화가 있고, 그 상한이 어디인지는 이 방법의 열린 질문이다.

품질은 놀랍도록 잘 버틴다. GPT-3 175B 에서 LoRA 4.7M 파라미터가 WikiSQL 73.4 / MNLI-m 91.7 이고 full fine-tuning 175,255.8M 이 73.8 / 89.5 다 [lora Table 4]. 학습 파라미터가 약 1/37,000 인데 한쪽에서는 오히려 앞선다. 다만 전승은 아니다 — QQP 에서는 두 크기 모두 LoRA 가 낮고, 가장 널리 인용되는 격차 하나는 warm-start 이점을 포함한 값이다 [lora Table 2]. 이 책은 LoRA 를 “full fine-tuning 과 대등한 쓰기 도구” 로 취급하되 “언제나 우월하다” 로는 취급하지 않는다.

손으로 세는 예제 — LoRA delta 는 정확히 몇 MB 인가

이 권에서 가장 실용적인 계산이다. 계산기가 필요한 곳은 없다.

$$|\Delta\Theta| = 2 \times N_{\text{mod}} \times d \times r, \quad N_{\text{mod}} = L_{\text{layer}} \times |\mathcal{S}|$$

기호 풀이 $|\Delta\Theta|$ = delta 아티팩트가 담고 있는 파라미터의 개수. d = 모델 폭. L_{layer} = 층 수. $\mathcal{S}$ = LoRA 를 걸어둔 weight 행렬의 종류 집합 (예: attention 의 query 와 value 두 종류면 $|\mathcal{S}| = 2$). N_{mod} = 그래서 실제로 LoRA 가 붙은 행렬의 총 개수. 계수 2 는 A 와 B 두 장에서 온다.

이 식이 말하는 것은 delta 의 크기가 **모델의 파라미터총수와 무관하다**는 것이다. 폭 · 깊이 · rank · 적용 집합 네만이크기를 정한다.

1 단계 — 네 값을 고정한다. dense-8B 앵커를 쓴다: $d = 4096$, $L_{\text{layer}} = 32$. 적용 집합은 query 와 value 두 종류, rank 는 $r = 16$, 저장 dtype 은 bf16 으로 둔다.

2 단계 — 행렬 개수. $N_{\text{mod}} = 32 \times 2 = 64$. 층마다 두 장씩, 32 개 층이므로 64 장이다.

3 단계 — 파라미터 수. $2 \times 64 = 128$. $128 \times 4096 = 524,288$. $524,288 \times 16 = 8,388,608$. 즉 **8.39M** 파라미터이고, 8B 모델 대비 약 10^{-3} 이다.

4 단계 — 바이트. bf16 은 원소당 2 byte 이므로 $8,388,608 \times 2 = 16,777,216 \text{ byte} = 16.78 \text{ MB}$ 다. int8 이면 8.39 MB, int4 면 4.19 MB 다.

교차 검증. 같은 식에 GPT-3 을 넣는다: $d = 12,288$, $L_{\text{layer}} = 96$, 같은 두 종류, $r = 4$. $N_{\text{mod}} = 192$, $2 \times 192 = 384$, $384 \times 12,288 = 4,718,592$, $\times 4 = 18,874,368$ — 약 **18.9M** 이다. 논문이 같은 설정을 “18M parameters, roughly 35MB if stored in FP16” 이라고 적은 것과 파라미터 수에서 맞는다 [lora § 7.1]. 350GB 가 35MB 가 되었으니 약 10,000 배다.

두 knob 의 감도. r 만 16 에서 64 로 올리면 4 배 (67.11 MB), 적용 집합만 두 종류에서 네 종류로 늘리면 2 배 (33.55 MB), 둘 다 키우면 8 배 (134.22 MB) 다. 두 knob 은 곱으로 들어간다.

사용자 규모로 옮기면. 16.78 MB 를 사용자 10^6 명에 곱하면 웹 스토리지 16.78 TB 이고, int4 로 내리면 4.19 TB 다. **dtype 하나가 스토리지 계층 규모를 4 배음직이다.**

시스템 모델링 관점 merge 하면 GEMM 모양이 원래 모델과 글자 그대로 같아진다. 층이 늘지도, 문맥이 줄지도 않으므로 이 절차가 wake 에 더하는 지연 — 이 책은 그 값을 L_w 로 쓴다 — 이 정확히 0 이다. 그런데 merge 된 가중치 한벌은 정확히 한 사용자에게만 유효하다. 즉 merge 는 $L_w = 0$ 을 **shared-weight batching** 과 맞바꾼 것이다. 반대로 delta 를 합치지않고 따로 태우면 배치는 지킬 수 있다 — 이때 추가되는 토큰당 연산은 base 대비 $2r/d = 32/4096 \approx 0.78\%$ 로 무시할 수준이다. 대신요청마다 수 MB 의 delta 를 로드해야 하고, 그 지연을 측정한 논문은 이 corpus 에 없다. **경로 사이의 가장 큰 비용 차이는 정확도가 아니라 배치 가능성에서 나온다.**

주의 자주 혼동되는 두 축을 갈라 둔다. QLoRA 는 열려 둔 Θ_0 를 4-bit 로 양자화하는 것이며 줄이는 대상은 **학습 시점의 GPU 메모리**다. 양자화된 delta 는 $\Delta\Theta$ 자체를 낮은 dtype 으로 저장하는 것이며 줄이는 대상은 **보관 · 전송바이트**다. 위 4 단계의 int4 가 후자다. 두 절차는 이름이 닮았을 뿐 바꾸는칸이 다르다.

자세한 것은 ch03 § 03.3·§ 03.5·§ 03.6.

10. distillation — 정답 자리에 다른 모델의 분포를놓는다

§ 9 까지가 “ Θ 의 어디에 쓸 것인가” 였다면 이 절은 “무엇을 목표로 쓸 것인가” 다. **distillation** 은 loss 의 정답 자리에 사람이 붙인 label 대신 다른 모델의 출력 분포를 놓는 절차다. 이 치환 하나가 전부다.

기호 풀이 **teacher** = 학습 중 파라미터가 움직이지 않고 그 출력이 학습신호로 소비되는 모델. **student** = 그 신호를 재현하도록 파라미터가 움직이는 모델. 두 이름은 **역할**이지 개체가 아니다 — 크기도구조도 정체성도 정의에 들어가지 않는다. 그래서 teacher 와 student 가 같은모델일 수도 있고, teacher 가 student 보다 **작을** 수도 있다. 이 절 뒤쪽이 그 두 경우를 차례로 다룬다.

왜 label 대신 분포인가. 정답 label 은 어휘 위의 one-hot 벡터라서 정답토큰 하나에 대해서만 정보를 준다. teacher 의 출력 분포는 같은 자리에서어휘 전체에 값을 준다 — “정답은 A 이고, B 는 A 의 8 분의 1 만큼 그럴듯하며, C 는 사실상 불가능하다” 는 순위와 간격까

지 함께 온다. 같은 토큰 하나를 소비하고도 student가 받는 신호의 차원이 1에서 어휘 크기로 늘어난다. 이여분의 좌표들을 **dark knowledge**라고 부른다.

비유 정답지는 정답 하나만 알려주고, 선생의 채점표는 오답들의 순위까지 알려준다. 같은 문제 하나에서 후자가 훨씬 많은 것을 가르친다. **이비유가 깨지는 곳** — 선생이 틀리면 학생은 그 틀림까지 정확히 배운다. 더 근본적으로, 아래에서 볼 self-distillation에서는 선생이 곧 학생이므로 “더 아는 선생”이라는 그림 자체가 성립하지 않는다.

temperature. dark knowledge를 얼마나 크게 들려줄지를 정하는 다이얼이 하나 있다. **temperature** T 는 softmax에 넣기 전 logit을 나누는 양수 스칼라다 (이 권에서 T 는 오직 이 뜻이며, 시퀀스 길이는 §3에서 정한 대로 L 이다). T 를 키우면 분포가 평평해져 낮은 확률의 좌표들이 상대적으로 커지고, $T \rightarrow 0$ 이면 argmax 하나만 남는다. 즉 T 는 **teacher의 어느 부분을 학습 신호로 삼을지 고르는 손잡이다**. 크기를 감각하려면 숫자 하나면 된다 — teacher logit이 (4, 2, 0)일 때 세 번째 좌표의 확률은 $T = 1$ 에서 0.0159이고 $T = 4$ 에서 0.1863이다. **11.7배**다. 다만 T 를 바꾸면 loss의 눈금 자체가 바뀌므로 hard label과 soft target을 섞는 비율도 함께 움직인다. 그래서 표준형에는 T^2 보정 상수가 붙는다.

시스템 모델링 관점 distillation은 새로운 커널을 하나도 도입하지 않는다. teacher 쪽은 순수 forward — 우리가 매일 돌리는 batch prefill과 완전히 같은 연산이고 가중치는 read-only다. student 쪽만 backward가 붙는다. 새로 생기는 것은 연산이 아니라 **두 모델이 동시에 HBM에 상주한다**는 배치 제약, 그리고 teacher 출력을 student backward가 끝날 때까지 살려 두어야 한다는 텐서 수명이다. 크기를 세어 보면 이것이 왜 제약인지 보인다: 어휘 128,000, 시퀀스 길이 2,048, bf16이면 teacher logit 텐서 하나가 $2,048 \times 128,000 \times 2B = 500$ MiB다. **시퀀스 한 개당** 그렇다. 8B 모델의 bf16 가중치가 16 GB이므로 시퀀스 하나의 logit이 이미 가중치의 약 3%다. 상위 64개 좌표만 남기면 768 KiB로 **666.7배** 작아지지만, 잘려 나간 것이 정확히 방금 T 로 키운 그 꼬리다. **temperature로 꼬리를 키우는 일과 top- k 로 꼬리를 버리는 일은 같은 파이프라인 안에서 서로를 상쇄한다**. 두 값을 따로 정하면 안 된다.

self-distillation. teacher와 student가 같은 모델일 때다. 여기서 즉시 나오는 질문이 이 절의 핵심이다 — teacher가 student보다 아는 것이 없는데 어떻게 이득이 나는가. 답은 셋이고 셋 다 “지식이 이동해서” 가아니다.

- **표본 출처가 바뀐다.** 남이 쓴 문장 위에서만 배운 모델은 자기가 쓴 문장 위에서 처음 보는 상태에 놓인다. 학습 분포와 추론 분포의 불일치다. student가 스스로 디코딩한 토큰 위에서 점수를 매기면 그 불일치가 사라진다. 표본을 고정 데이터셋에서 뽑으면 **off-policy**, student 자신의 분포에서 뽑으면 **on-policy**다.
- **목표가 label보다 부드럽다.** one-hot label은 정답 아닌 좌표를 전부 0으로 밀지만 자기 분포는 그렇지 않다. 정칙화로 작동한다.
- **teacher가 앵커가 된다.**昨天的 자기를 목표로 두면 오늘의 갱신은 옛 행동에서 멀어지는 것을 loss로 벌받는다. 옛 데이터를 다시 넣는 것과 목적은 같고 기제는 다르다 — 데이터가 아니라 함수를 붙잡는다.

핵심 on-policy 비율이 0보다 커지는 순간 sleep job의 **비용 형태**가 바뀐다. 0이면 sleep은 평범한 training job이고 지배 커널은 GEMM이다. 0보다 크면 매 갱신 앞에 student의 autoregressive 생성이 붙으므로 지배 커널이 decode가 된다. §7의

결론이 여기서 다시 나온다 — sleep job 은 wake 와 같은 병목을 놓고 경쟁한다.

upward distillation. 마지막으로 이 라인이 실제로 쓰는 형태하나를 본다. teacher가 student보다 파라미터가 적은 distillation이다. 방향이 반대다. 통상의 정당화는 “teacher가 더 많이 알고 student는 답을 수 있는 만큼 답는다” 인데, 여기서는 teacher가 덜 안다.

핵심은 옮겨지는 것이 정보량이 아니라 좌표라는 것이다. 절차는이렇다. 먼저 모델에 작은 파라미터 한 벌을 새로 붙인다. 이 확장 직후의 모델은 확장 전과 다른 함수다 — 새 파라미터의 값이 무엇이든 그것이 forward에 더해지는 순간 출력이 흔들린다. 즉 용량을 늘리는 행위 자체가 손상이다. upward distillation은 그 손상을 되돌린다. teacher는 확장 전의 자기, student는 확장 후의 자기이고, 학습 대상은 새로붙인 파라미터뿐이다. 성공하면 새 파라미터는 “무해하면서 동시에 옛 행동을 자기 좌표로 표현할 줄 아는” 상태가 된다. 새로 생긴 것은 지식이 아니라 비어 있고 정렬된 용량이다.

주의 이 절차의 기여 크기는 논문 자신의 표에서 작다. Sleep이 최강 baseline을 이기는 마진이 +2.6 / +1.6 / +1.0 인데, 그중 +1.3 / +0.8 / +0.8은 이 seeding 없이 구조적 확장만으로 얻어진다 [LM Need Sleep Table 1]. 같은 표에서 한 구성요소를 빼면 점수가 오히려 오르는 행도 있다. 그런데 논문 본문은 “모든 구성요소가 성능에 긍정적으로 기여한다” 고 쓴다. 시드도 신뢰구간도 없다. 이 라인의 논문을 읽을 때 본문 문장보다 ablation 표를 먼저 보아야 하는 이유다.

자세한 것은 ch04 § 04.1 – § 04.5.

11. 한 장으로 — 무엇이 갱신되고 얼마나 드는가

이 권의 산출물이다. 어떤 논문을 읽든 그 논문의 학습 절차를 이 표의 한행으로 적을 수 있어야 한다.

표 2 — 레짐과 도구의 네 칸

절차	무엇이 데이터인가	무엇을 목표로 하는가	무엇이 갱신되는가	얼마나 드나
pre-training	주어진 대규모 코퍼스	다음 토큰 맞히기	Θ 전체	$6ND$. epoch $\approx$ 1. 배포 전 1회
SFT	(입력, 출력) 쌍	출력 구간만의 다음 토큰 맞히기	Θ 전체 또는 부분 집합	$6ND$ 또는 $4ND$. D 가 pre-training보다 여러자릿수 작다
RLHF / RLVR	지금의 Θ 가 만든 rollout	채점점수의 기댓값	$\Theta(\text{policy})$	생성분 $2ND$ 가 지배
full fine-tuning	고정 dataset	임의	Θ 전체	아티팩트 = base와 동일 크기 (350GB)

절차	무엇이 데이터인가	무엇을 목표로 하는가	무엇이 갱신되는가	얼마나 드나
LoRA	고정 dataset	임의	rank r 부분공간	아티팩트 $2N_{\text{mod}}dr(8B \cdot r=16 \text{ 에서 } 16.78 \text{ MB})$
distillation	임의 (teacher 분포가 목표)	teacher 분포와의 거리	student 의 Θ	teacher forward + logit 저장. on-policy 면 decode 가 지배
유틸 시간 갱신	누적 경험에서 모델이 만든다	라운드마다 다름	Θ 의 지정 부분 집합	라운드당 예산. FLOPs 로 보고한 논문이 거의 없다

마지막 행의 마지막 칸이 이 책이 반복해서 부딪히는 공백이다. 레짐의 이름과 하이퍼파라미터는 표로 실리지만 그것이 몇 FLOPs 인지, 라운드가 몇번 도는지는 대부분의 논문이 적지 않는다. § 3 의 두 식이 그 칸을 복원하는 도구다.

자세한 것은 ch02 § 02.8.

12. 네 개의 질문 — “이 값은 누가학습하는가”

레짐 어휘를 배우는 목적은 분류 자체가 아니라 **판정 습관**이다. 논문에 등장하는 임의의 값 하나를 골라 네 질문을 순서대로 던진다.

1. **그 값은 배포 아티팩트에 남는가?** 남으면 Θ 이거나 Θ 의 증분이다. 세션과 함께 사라지면 W 이거나 E 다.
2. **그 값을 움직이는 gradient 는 어느 loss 의 것인가?** 요청을 처리하는 도중 토큰마다 계산되는 작은 loss 면 W , § 2 의 학습 loss 면 Θ , gradient 가 아예 없으면 E 다.
3. **어느 시계에서 움직이는가?** 토큰마다인가, optimizer step 마다인가, 유틸 라운드마다인가, 세션마다인가.
4. **그 값을 만든 데이터는 주어진 것인가, 모델이 만든 것인가?** 후자면 자기가 만든 데이터로 자기를 학습시킬 때의 붕괴 위험이 이 논문에 걸린다.

앞의 셋만으로도 세 서랍이 갈린다. 넷째는 그 위에 얹히는 정직성 점검이다.

핵심 이 습관의 값어치는 예제 하나로 드러난다. “sleep-time compute” 이라는 이름을 만든 논문에 이 질문들을 던지면 Θ 쪽 답이 하나도 나오지 않는다 — 가중치 공간의 상태를 쓰지 않는다고 그 논문이 스스로 밝히고, 남는 것은 E 서랍뿐이다 [Letta STC § 7]. 반면 제목에 “sleep” 을 단 다른 논문은 답이 Θ 쪽이다. **같은 이름 아래 완전히 다른 서랍을 연다는 사실이 이 네 질문만으로 즉시 드러난다.** 이 시리즈를 읽는 내내 이 판정을 먼저 하고 그다음 내용을 읽어야 한다.

자세한 것은 ch02 § 02.10, 판별식의 정본은 ch11.

요약 - 이 라인이 쓰는 자리는 세 서랍뿐이다: Θ (checkpoint, 배포아티팩트가 바뀜) · W (세션 수명의 빠른 상태) · E (모델 바깥의 저장소). training은 Θ 서랍전용의 절차다. - 학습은 손잡이를 gradient 반대 방향으로 보폭 η_Θ 만큼 옮기는 일의 반복이고, 이 문법은 pre-training 이든유티 시간 갱신이든 글자 그대로 같다. 레짐을 가르는 것은 데이터 출처 · 목표 · 움직이는 부분집합 셋뿐이다. - 비용의자는 두 식이다: $D = n_{\text{step}} \times N_b \times L$, 그리고 $6ND(\text{학습}) / 4ND(\text{부분 갱신}) / 2ND(\text{생성만})$. decode 한 토큰을 $2N$ 으로 잡던 감각에 인자 3 을 붙이면끝이다. - training 이 HBM 을 먹는 이유의 절반은 optimizer 상태다. 그래서학습 파라미터를 줄이면 메모리는 파라미터 수보다 가파르게 줄어든다. - RL 레짐의 정의적 특징은 보상이 아니라 데이터가 Θ 의 함수라는것이고, 그 결과 한 job 안에서 decode 성격과 prefill 성격이 교대한다. 유티 시간 갱신이 이 형태를 물려받으면 sleep 은 wake 와 같은 병목을 놓고경쟁한다. - full fine-tuning 은 곧 모델 재배포다 (GPT-3 에서 task 마다 350GB). LoRA delta 의 크기는 $2N_{\text{mod}}dr$ 로 정확히 주어지며모델 파라미터 총수와 무관하다 — $8B \cdot r=16 \cdot \text{bf16}$ 에서 **16.78 MB**, 사용자 10^6 명이면 16.78 TB, int4 면 4.19 TB 다. - merge 는 wake 지면 0 을 shared-weight batching 과 맞바꾼 것이다. 경로 사이의 가장 큰 비용 차이는 정확도가아니라 배치 가능성에서 나온다. - distillation 은 gradient 의 출처를 label 에서 다른 모델의 분포로 바꾸는 절차이고, teacher/student 는 크기가 아니라 **파라미터의 이동여부**로 정의된다. 그래서 “작은 teacher, 큰 student” 가 형용모순이아니다. - SFT 를 돌렸다는 사실은 능력이 올랐다는 증거가아니다 (Qwen3-1.7B 에서 AIME-24 가 49.8 $\rightarrow$ 47.3). 본문 문장보다 ablation 표를 먼저 본다. - 임의의 값에 네 질문을 던져 서랍을 판정하는 습관이 이권의 진짜 산출물이다.

다음 권 예고 — E02 에서는 이 권이 비워 둔 칸을 채운다. § 11 표 2 의 마지막 행에서 데이터가 “모델이 만든다” 고 적힌 자리다. 모델이 자기학습 데이터를 만들 때 무엇이 좋아지고 무엇이 무너지는가, 옛 지식이 새라운드에서 지워지는 현상은 어떤 모양인가, 그리고 그것을 막는 replay 와 consolidation 은 정확히 무엇을 되풀이하는가. 이 권에서 Θ 에쓰는 법을 익혔다면, 다음 권은 **반복해서 쓸 때 벌어지는일을** 다룬다.